

LEARNING RESOURCE GUIDE

ISC2 CCSP: Certified Cloud Security Professional (Aug 2026) | Professional Participant Workbook

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- By the end of this lecture, you will be able to identify the five cloud-specific risks that a Secure SDLC must explicitly address when applied to cloud application development.
- By the end of this lecture, you will be able to explain why a standard on-premises SDLC cannot be applied to cloud application development without modification.
- By the end of this lecture, you will be able to describe the shared responsibility model as the starting point for defining the security scope a development team must address.
- By the end of this lecture, you will be able to apply cloud-specific risk awareness to the design and requirements phases of a Secure SDLC.
- By the end of this lecture, you will be able to articulate the regulatory and jurisdictional complexity that cloud application development introduces and how it must be addressed.

Introduction

A development team that follows a documented, disciplined Secure SDLC and still ships a vulnerable cloud application is not an unusual scenario. It happens because many teams apply a development process designed for on-premises software to a cloud environment without accounting for the risks that are unique to how cloud platforms work. The cloud is not simply a different data centre. It is a structurally different operating environment with a different threat surface, different visibility constraints, and a fundamentally different model for who controls what. A Secure SDLC that ignores these differences is a process with predictable, exploitable gaps.

This lecture addresses five cloud-specific risks that the CCSP Domain 4 curriculum identifies as requiring explicit treatment in any Secure SDLC applied to cloud application development: shared technology vulnerabilities, Cloud Service Provider insider threats, lack of visibility and audit capability, supply chain risk from cloud-native services and dependencies, and jurisdiction and regulatory risk. Understanding these risks is not merely academic. Each one has produced real-world incidents where organisations building on cloud platforms discovered — often through breaches — that their development process had not accounted for a risk that was known and manageable from the start.

The practical tool for understanding which risks are yours to manage is the shared responsibility model. Every major Cloud Service Provider publishes a shared responsibility model that defines which security controls the CSP owns and which are the customer's responsibility. That model is the starting point for defining the security scope that a development team must address in its Secure SDLC. Failing to read and internalise the shared responsibility model before the requirements phase is one of the most common and consequential mistakes teams make when building cloud applications.

Core Concepts Explained

Shared Technology Vulnerabilities

Definition: Cloud platforms are built on shared infrastructure — hypervisors, container orchestration systems, hardware acceleration layers, and shared networking fabric. A vulnerability in any shared component can potentially affect every tenant on that platform, regardless of the security of the individual application.

Business relevance: Shared technology vulnerabilities are outside the development team's control but inside their threat model. Applications that assume the cloud platform is fully trusted and impose no controls of their own are maximally exposed if a shared infrastructure vulnerability is exploited.

Cloud Security Practitioner relevance: Cloud Security Practitioners design applications to limit the impact of a shared technology compromise: encrypting data at rest and in transit so that a hypervisor escape cannot read plaintext data, enforcing least privilege so that lateral movement within a compromised environment is constrained.

Real-world example: The Spectre and Meltdown vulnerabilities disclosed in January 2018 demonstrated the real-world impact of shared technology risk at cloud scale. Both vulnerabilities exploited speculative execution in Intel, AMD, and ARM processors — the physical hardware underlying virtually every major cloud platform. An attacker who could exploit Meltdown could potentially read memory across virtual machine boundaries on the same physical host, directly threatening the tenant isolation that cloud platforms rely on. AWS, Azure, and GCP all implemented emergency mitigations within days, but the incident illustrated that no cloud tenant can assume the isolation boundary is absolute. Development teams that had implemented application-layer encryption — encrypting sensitive data before it reached the hypervisor, using customer-managed keys — maintained confidentiality even in environments where the shared technology layer was vulnerable. Teams that relied solely on CSP-managed encryption and platform isolation had no additional control layer to fall back on.

Risks or limitations: The risk of ignoring shared technology vulnerabilities is that an application's security posture depends entirely on the CSP maintaining a flawless isolation boundary — an assumption that the Spectre/Meltdown class of vulnerabilities proved is not guaranteed. The best practice is implementing application-layer encryption with customer-managed keys so that data confidentiality does not depend solely on the CSP's infrastructure isolation.

CSP Insider Threat

Definition: When an application is hosted on a cloud platform, the CSP's employees have physical and logical access to the underlying infrastructure. For sensitive data workloads, this introduces a threat model that on-premises teams rarely needed to consider.

Business relevance: CSP insider threat is a managed and generally low-probability risk for most workloads, but for highly sensitive data — financial records, health information, classified government data — it requires specific controls. Failure to address it can also create compliance gaps, as regulations such as HIPAA require controls against unauthorized access by any party, including cloud providers.

Cloud Security Practitioner relevance: Cloud Security Practitioners implement controls that ensure sensitive data cannot be read in plaintext by CSP administrators: customer-managed encryption keys stored in HSMs the CSP cannot access, confidential computing environments that encrypt data in use, and contractual controls such as data processing agreements that limit CSP employee access.

Real-world example: A financial services firm building a trading platform on Google Cloud Platform handled customer portfolio data that was subject to both SEC regulations and client confidentiality agreements. When

the security architect assessed the CSP insider threat risk during the design phase, they determined that standard GCP server-side encryption — where Google manages the keys — was insufficient: Google employees with sufficient access could theoretically decrypt the data. The architect implemented Google Cloud External Key Manager (Cloud EKM), which stored the encryption keys in the firm's on-premises HSM rather than in Google's key management infrastructure. Under this configuration, GCP could not decrypt the data without calling out to the firm's HSM, which logged every decryption request. A GCP employee with infrastructure access could see encrypted data but not plaintext. The firm disclosed this control to its clients and regulators as evidence that CSP insider access was architecturally constrained.

Risks or limitations: The risk of ignoring CSP insider threat is regulatory non-compliance and reputational exposure for workloads where client or regulatory expectations require that only the customer can access data in plaintext. The best practice is classifying data by sensitivity and applying customer-managed key management for data classifications where CSP administrator access to plaintext is not acceptable.

Lack of Visibility and Audit Capability

Definition: In cloud environments, the level of visibility available to the application team depends on what the CSP exposes through its logging and monitoring services. Unlike a traditional data centre where teams can install monitoring tools on physical hardware, cloud visibility must be deliberately configured through CSP-provided services.

Business relevance: An application that does not generate sufficient logs and alerts cannot support effective incident response, compliance auditing, or anomaly detection. Compliance frameworks such as PCI DSS, HIPAA, and SOC 2 all require audit trails for data access events that must be explicitly configured in cloud environments.

Cloud Security Practitioner relevance: Cloud Security Practitioners make logging and observability first-class security requirements during the Secure SDLC requirements phase, specify the audit events the application must generate, configure CSP logging services (CloudTrail, Azure Monitor, GCP Cloud Audit Logs), and establish alert policies that surface anomalous activity in near-real time.

Real-world example: A healthcare SaaS company operating on AWS had configured their application to log application-layer errors and user authentication events but had not configured CloudTrail for S3 data access events or AWS Config for infrastructure change tracking. When a security incident occurred — an IAM access key was compromised and used to access patient records stored in S3 — the incident response team could not determine which records had been accessed, when, or by what API calls, because S3 data access logging had not been enabled. The regulatory impact was significant: HIPAA's Breach Notification Rule requires organisations to determine the scope of a breach, and the inability to produce access logs meant the organisation had to assume worst-case exposure and notify all potentially affected patients. The post-incident remediation included enabling S3 server access logging, CloudTrail for all S3 API calls, and establishing CloudWatch alarms for anomalous access patterns — all controls that should have been Secure SDLC requirements from day one.

Risks or limitations: The risk of insufficient cloud visibility is that incidents cannot be detected, their scope cannot be determined, and regulatory obligations for audit trails cannot be met. The best practice is treating logging and observability as first-class security requirements in the SDLC requirements phase, specifying them at the same level of detail as functional requirements.

Supply Chain Risk from Cloud-Native Services

Definition: Cloud applications rarely stand alone — they consume managed database services, serverless functions, API gateways, machine learning APIs, authentication services, and content delivery networks

provided by the CSP or third-party vendors. Each integration extends the attack surface and introduces a dependency whose security posture the application team does not fully control.

Business relevance: Supply chain attacks against software dependencies have become one of the most significant threat vectors in cloud application security. The SolarWinds and Log4Shell incidents demonstrated that widely trusted components can harbour vulnerabilities that propagate to every application that depends on them.

Cloud Security Practitioner relevance: Cloud Security Practitioners conduct security assessments of each cloud service dependency, evaluate the CSP's or vendor's security posture for that service, apply least privilege to every service-to-service integration, and maintain an up-to-date software bill of materials (SBOM) that enables rapid identification of affected services when a dependency vulnerability is disclosed.

Real-world example: The Log4Shell vulnerability (CVE-2021-44228), disclosed in December 2021, affected the Apache Log4j2 Java logging library — a component present in thousands of applications and cloud services. Organisations with a maintained SBOM were able to identify which of their cloud applications used Log4j2 within hours of the CVE publication. Organisations without an SBOM spent days or weeks manually auditing their codebases and dependency trees. One cloud application team discovered that their application did not directly import Log4j2 but that their AWS Elasticsearch Service (now OpenSearch) used it internally — a supply chain dependency they had not tracked. AWS released a patched version within days, but teams that had not modelled their third-party service dependencies as part of their Secure SDLC supply chain review were caught by surprise by a risk they had no process to manage.

Risks or limitations: The risk of unmanaged supply chain dependencies is that a vulnerability in any dependency — whether a library the team wrote code against or a managed cloud service they consume — can propagate to the application without warning. The best practice is maintaining a software bill of materials for all dependencies, including managed cloud services, and subscribing to CSP security advisories as part of the maintenance-phase vulnerability management programme.

Jurisdiction and Regulatory Risk

Definition: Cloud platforms are geographically distributed. Data stored in a cloud environment may reside in multiple regions across multiple countries, subject to the legal frameworks of each jurisdiction. Regulatory requirements — data residency, data sovereignty, cross-border transfer restrictions — must be addressed as design inputs, not post-deployment compliance documentation.

Business relevance: Regulatory non-compliance resulting from data stored in the wrong jurisdiction can carry significant financial penalties — GDPR fines up to 4% of annual global revenue — and in some cases legal liability for individuals. Discovering a regulatory conflict after deployment is more expensive and disruptive than designing for it from the start.

Cloud Security Practitioner relevance: Cloud Security Practitioners identify all applicable regulatory jurisdictions during the Secure SDLC requirements phase, specify data residency and sovereignty requirements that flow into architecture decisions (region selection, data replication policies, cross-border transfer mechanisms), and validate compliance before the application is deployed.

Real-world example: A German financial services company built a customer data analytics platform on AWS with a team that understood GDPR generally but had not mapped specific architectural decisions to GDPR requirements. During a compliance review six months after deployment, the firm's data protection officer discovered that AWS's default cross-region replication feature had been enabled, causing customer transaction data to replicate automatically to an AWS region in the United States. Under GDPR Chapter V, transferring personal data to the United States requires either adequacy decisions, standard contractual clauses, or binding corporate rules — none of which the firm had in place for this data flow. The remediation required disabling

cross-region replication, migrating data back to EU regions, implementing data residency controls, and establishing Standard Contractual Clauses with AWS as the data processor — all work that would have taken hours to design in at the requirements phase but required months to remediate after deployment.

Risks or limitations: The risk of ignoring jurisdictional complexity is that data may flow to jurisdictions where the organisation lacks the legal basis for processing or transferring it. The best practice is treating data residency and regulatory jurisdiction as mandatory architecture constraints from the requirements phase, using CSP region controls and data residency features as design elements rather than post-deployment configurations.

Practical Applications

Each of the five cloud-specific risks described in this lecture requires concrete action during the Secure SDLC requirements and design phases — before development begins.

Assessing Cloud-Specific Risks During the Requirements Phase

A Cloud Security Practitioner joining a project team building a customer data platform on Microsoft Azure begins by reviewing the Azure shared responsibility model for the services the team plans to use. They identify which security controls Azure owns (physical security, hypervisor isolation, managed service patching) and which the customer owns (identity and access management, encryption key management, network configuration, application code security). This review establishes the security scope the development team must address and prevents the common mistake of assuming Azure handles controls that are actually the customer's responsibility. The practitioner then works through the five cloud-specific risk categories systematically: they specify a data residency requirement that all customer data remain in EU Azure regions, an encryption requirement that all customer PII be encrypted using customer-managed keys in Azure Key Vault, a logging requirement that all data access events generate audit records in Azure Monitor, a supply chain review requirement that all third-party integrations be security-assessed before sprint planning, and a CSP insider threat control requirement for the subset of data classified as confidential. These requirements enter the backlog before the design phase begins.

Designing Controls for CSP Visibility Limitations

A Cloud Security Practitioner is reviewing the design of a regulated financial application deployed on Google Cloud Platform. During the design review, they identify that the application team has not specified how audit events will be generated or stored. The practitioner raises this as a design-phase finding and works with the team to design a comprehensive logging architecture: GCP Cloud Audit Logs enabled for all admin activity and data access events, application-generated audit records for every user data access event written to a separate immutable Cloud Storage bucket with object lock enabled, Cloud Monitoring alert policies for anomalous access patterns (access outside business hours, access from unexpected IP ranges, bulk data export operations), and a log retention policy aligned to PCI DSS's 12-month log retention requirement. The practitioner specifies that these logging requirements must be implemented in the first sprint, not deferred to a later sprint, because the audit trail must exist from the first day of production operation — compliance auditors frequently ask for log data from the day the system was commissioned, not just recent data.

Managing Supply Chain Risk Through SBOM and Dependency Controls

A Cloud Security Practitioner establishes a supply chain risk management process for a cloud application team building a Java-based microservices platform on AWS. They require that every service maintain a Software Bill of Materials (SBOM) generated automatically from the Maven dependency tree and stored in the team's artefact

repository alongside each build. They configure AWS Inspector to scan all Lambda functions and container images for known CVEs at build time, and subscribe the team's security mailing list to AWS Security Advisories so that when a managed service vulnerability is disclosed, the team receives notification immediately. When a vulnerability is disclosed in a Spring Framework version used by three of the team's microservices, the SBOM enables the practitioner to identify the affected services within 15 minutes, assess whether the vulnerable code path is exercised, and open P1 remediation tickets with a 48-hour SLA. The practitioner also establishes a quarterly supply chain review process in which all third-party service integrations are reassessed against the CSP's published security documentation, confirming that the services' security posture has not materially changed since the initial design-phase assessment.

Best Practices

- **Start every cloud application project by reading the CSP's shared responsibility model.** The shared responsibility model defines which security controls the CSP owns and which are the customer's responsibility. Misreading or ignoring this model is the root cause of many cloud security gaps — teams assume the CSP handles controls that are actually their own responsibility. Read the shared responsibility model for every CSP service the application will use before the requirements phase is complete.
- **Treat data residency and regulatory jurisdiction as mandatory architecture constraints, not post-deployment configurations.** Specify which jurisdictions data may reside in, which regulatory frameworks apply, and what cross-border transfer mechanisms are in place during the requirements phase. Discovering a GDPR cross-border transfer violation after deployment requires months of remediation. Designing for it from the start requires hours. Region selection, data replication policies, and data processing agreements must be design decisions, not operational afterthoughts.
- **Implement application-layer encryption with customer-managed keys for all sensitive data classifications.** CSP-managed encryption protects data against external attackers but does not address CSP insider threat or shared technology vulnerabilities. Customer-managed keys in external HSMs or hardware security modules ensure that data confidentiality does not depend on the CSP's key management infrastructure or infrastructure isolation. Apply this control for all data classified as confidential or higher.
- **Make logging and observability first-class security requirements from the requirements phase.** Specify the audit events the application must generate, the CSP logging services that must be configured, and the alert policies that must be active before the system goes live. Applications that cannot answer 'who accessed what data and when' in response to an incident cannot meet their incident response obligations or regulatory audit requirements. Logging must be designed in, not retrofitted after an incident.
- **Maintain a Software Bill of Materials for all dependencies, including managed cloud services.** An SBOM enables rapid identification of affected components when a vulnerability is disclosed. Without one, teams spend days or weeks in manual audits during an incident response or following a major CVE disclosure. Generate the SBOM automatically from build tooling (Maven, pip, npm, etc.) and store it alongside each build artefact. Subscribe to CSP security advisories so that managed service vulnerabilities trigger the same response process as library CVEs.

Common Mistakes and Misconceptions

Misconception: Applying an on-premises SDLC to cloud application development without modification, assuming the same risks and controls apply.

Correct approach: Cloud environments introduce five structurally distinct risks — shared technology, CSP insider threat, limited visibility, supply chain dependencies, and jurisdictional complexity — that on-premises SDLCs do not address. The best practice is explicitly reviewing each of these risk categories during the requirements and design phases of any cloud application project.

Misconception: Assuming the CSP handles security controls that are actually the customer's responsibility, based on a surface-level reading of CSP marketing materials.

Correct approach: The shared responsibility model is the authoritative reference for which controls belong to the CSP and which to the customer. Read it for every CSP service the application will use. Key management, application-layer logging, network access controls, and identity management are almost always customer responsibilities, even when the CSP provides the underlying service.

Misconception: Treating penetration testing in a cloud environment as equivalent to penetration testing in a data centre, using the same tools and scope without CSP approval.

Correct approach: Cloud CSPs require customers to obtain approval before conducting penetration testing on their platforms, and prohibit certain testing types (network scanning at scale, denial of service testing) that are acceptable in a private data centre. The Secure SDLC must account for these constraints when scoping testing activities, and pen test plans must be reviewed against the CSP's acceptable use policy before the testing phase begins.

Misconception: Discovering jurisdictional and regulatory conflicts after deployment, when data is already residing in the wrong jurisdiction.

Correct approach: Data residency, sovereignty, and cross-border transfer requirements must be captured during the requirements phase and drive region selection, data replication policy, and data processing agreement decisions during the design phase. Remediation after deployment — migrating data, disabling replication, establishing legal bases for cross-border transfers — is orders of magnitude more expensive than designing for these requirements from the start.

table: Cloud-Specific Risk Assessment Checklist for Secure SDLC

Use this table during the requirements and design phases of any cloud application project to ensure each cloud-specific risk has been explicitly assessed and addressed.

Risk Category	Key Questions to Answer	Control to Design In
Shared Technology Vulnerabilities	What data would be exposed if the hypervisor or container isolation boundary were breached?	Application-layer encryption with customer-managed keys; least privilege for all service accounts
CSP Insider Threat	What data classifications require controls against CSP administrator access to plaintext?	Customer-managed keys in external HSM; data processing agreement limiting CSP staff access
Lack of Visibility	Which data access events must be logged? What alerts are required? What is the retention requirement?	CSP audit logging enabled for all data access; application-generated audit records; immutable log storage
Supply Chain Risk	What managed CSP services and third-party libraries does the application consume? Has each been security-assessed?	SBOM generated at every build; CSP advisory subscriptions; quarterly supply chain review
Jurisdiction and Regulatory Risk	Which jurisdictions will data reside in? Which regulatory frameworks apply? What cross-border transfer	Data residency controls enforced by region configuration; data processing agreements; Standard Contractual

	mechanisms are in place?	Clauses
Penetration Testing Constraints	Has the pen test scope been reviewed against the CSP's acceptable use policy? Is CSP approval required?	CSP approval obtained before testing; prohibited test types excluded from scope

How to use this: Complete this table during the requirements phase and revisit it at the design phase exit. Each row should have a documented answer and a control assigned to a specific team member before development begins.

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

Q1. A development team building a cloud application discovers six months after deployment that AWS's default cross-region replication has been replicating European customer data to US-based regions, violating GDPR cross-border transfer requirements. Which Secure SDLC failure does this scenario most directly illustrate?

- A) A. Failure to conduct penetration testing before deployment.
- B) B. Failure to treat jurisdictional and regulatory requirements as architecture constraints during the requirements and design phases.
- C) C. Failure to implement application-layer encryption for customer data.
- D) D. Failure to configure CSP audit logging for data access events.

Answer: B. Failure to treat jurisdictional and regulatory requirements as architecture constraints during the requirements and design phases.

Q2. The Spectre and Meltdown vulnerabilities demonstrated that cloud hypervisor isolation boundaries can be compromised by hardware-level vulnerabilities. Which application-level control best protects data confidentiality in the event that a shared technology vulnerability allows cross-tenant memory access?

- A) A. Network segmentation using cloud VPCs to isolate tenant workloads.
- B) B. Application-layer encryption using customer-managed keys, ensuring data is encrypted before it reaches the hypervisor layer.
- C) C. Deploying applications in dedicated host instances to eliminate shared hardware.
- D) D. Enabling CSP-managed encryption at rest for all storage services.

Answer: B. Application-layer encryption using customer-managed keys, ensuring data is encrypted before it reaches the hypervisor layer.

Q3. Following a security incident, a cloud application team cannot determine which patient records were accessed by a compromised IAM credential because S3 data access logging was not enabled. Which Secure SDLC phase failure is primarily responsible for this gap?

- A) A. Testing phase failure — the penetration test did not include IAM credential compromise scenarios.
- B) B. Requirements phase failure — logging and observability were not specified as security requirements before development began.
- C) C. Deployment phase failure — the deployment checklist did not include enabling S3 logging.
- D) D. Maintenance phase failure — the security monitoring programme did not detect the credential compromise.

Answer: B. Requirements phase failure — logging and observability were not specified as security requirements before development began.

Q4. A cloud application team discovers that their platform uses a managed AWS service that internally relied on Log4j2, which was affected by the Log4Shell vulnerability. The team had not modelled this service as a supply chain dependency. Which control would most directly have enabled faster response to this vulnerability disclosure?

- A) A. Performing a penetration test of the managed service before incorporating it into the application.
- B) B. Maintaining a Software Bill of Materials (SBOM) that includes managed cloud service dependencies, enabling rapid identification of affected components.
- C) C. Implementing network segmentation between the application and the managed service.
- D) D. Encrypting all data exchanged with the managed service using customer-managed keys.

Answer: B. Maintaining a Software Bill of Materials (SBOM) that includes managed cloud service dependencies, enabling rapid identification of affected components.

Q5. A Cloud Security Practitioner is reviewing a Secure SDLC for a new cloud application and notes that the penetration testing plan includes network port scanning across all IP ranges in the AWS VPC. Which cloud-specific SDLC consideration requires modification of this plan?

- A) A. VPC network scanning is not technically effective in cloud environments and should be replaced with application-layer testing.
- B) B. Cloud CSPs require customer approval for penetration testing activities and prohibit certain test types, including large-scale network scanning; the plan must be reviewed against AWS's acceptable use policy before testing begins.
- C) C. Penetration testing should only be conducted on the production environment, not the VPC network.
- D) D. Network scanning is a CSP responsibility under the shared responsibility model and should not be performed by the application team.

Answer: B. Cloud CSPs require customer approval for penetration testing activities and prohibit certain test types, including large-scale network scanning; the plan must be reviewed against AWS's acceptable use policy before testing begins.

Reflection Questions

1. The lecture describes a development team that followed a documented SDLC but shipped a cloud application with a misconfigured public storage bucket. In your experience, what organisational dynamics lead to teams following process documents without internalising the risks those processes are designed to address?
2. The shared responsibility model defines what security controls belong to the CSP and what belongs to the customer. Have you reviewed the shared responsibility model for a cloud service you use? What surprised you about where the boundary falls?
3. Log4Shell demonstrated that supply chain vulnerabilities can propagate through managed cloud services the development team does not control. How would your current team respond if a critical CVE were disclosed in a managed service your application depends on? What would you want to have in place before that happens?

4. Jurisdictional complexity — data residing in multiple countries, subject to different legal frameworks — is described as a design input, not a compliance afterthought. How does this framing change the conversation about cloud architecture decisions, particularly region selection and data replication?
5. The lecture argues that cloud-specific risk does not make cloud development more dangerous than on-premises development — it makes it differently shaped. Do you agree? Are there risks in on-premises development that cloud development reduces or eliminates?

Key Takeaways

- A standard on-premises Secure SDLC cannot be applied to cloud development without modification — the cloud introduces five structurally distinct risks that require explicit treatment.
- The five cloud-specific risks are: shared technology vulnerabilities, CSP insider threat, lack of visibility and audit capability, supply chain risk from cloud-native services, and jurisdiction and regulatory risk.
- The shared responsibility model is the starting point for every cloud development project — it defines which security controls the CSP owns and which are the customer's responsibility.
- Application-layer encryption with customer-managed keys provides protection against both shared technology vulnerabilities and CSP insider threat, and is the appropriate control for sensitive data classifications.
- Logging and observability must be specified as first-class security requirements in the SDLC requirements phase — not configured as an afterthought during deployment or following an incident.
- Maintaining a Software Bill of Materials for all dependencies, including managed cloud services, enables rapid response when a supply chain vulnerability is disclosed.
- Jurisdictional and regulatory requirements — data residency, sovereignty, cross-border transfer mechanisms — must be captured during the requirements phase and must drive architecture decisions during design, not post-deployment remediation.

Recommended Next Actions

Put this lecture to work within the next seven days:

6. Review the shared responsibility model for the cloud platform your organisation uses. Identify three security controls you assumed the CSP handled that are actually the customer's responsibility. Document these and share them with your development team this week.
7. Check whether your cloud application has logging enabled for all data access events in your CSP's logging service (CloudTrail, Azure Monitor, GCP Cloud Audit Logs). If not, raise a backlog item to enable and configure it, specifying the events, retention period, and alert policies required.
8. Generate a Software Bill of Materials for one production cloud application using build tooling (mvn dependency:tree, pip list, npm list). Review it for any dependencies with active CVEs in the NVD, and document a triage and remediation process for findings.
9. Identify the regulatory jurisdictions applicable to one cloud application your team operates. Verify that data residency controls are in place and that the data processing agreement with your CSP addresses the applicable regulatory requirements. If gaps exist, document them as a compliance risk for your organisation's risk register.